

Punto 4.1 - Análisis del SQL generado: ToQueryString y logging

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que devuelvan el SQL generado por diferentes consultas. Configurar el logging de EF Core. Analizar el SQL de consultas con Where, OrderBy, Select e Include. Diagnosticar un problema de consulta ineficiente a partir del SQL generado.

Contexto del proyecto: En el punto 3.12 se consolidaron las buenas prácticas de acceso a datos, cerrando el Módulo 3. En este punto se inicia el Módulo 4 con el análisis del SQL generado, que es la base de la optimización. Esta técnica se usará en el punto 4.2 para el control del tracking y en el punto 4.3 para las variantes de No Tracking.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir métodos de análisis de SQL a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    string ObtenerSqlConsultasPendientes();
    string ObtenerSqlConsultasConInclude();
    string ObtenerSqlConsultasConProyeccion();
    string ObtenerSqlConsultasConFiltroGlobal();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: string ObtenerSqlConsultasPendientes(); → método que devuelve el SQL de las consultas pendientes.

Línea 11: string ObtenerSqlConsultasConInclude(); → método que devuelve el SQL de las consultas con Include.

Línea 12: string ObtenerSqlConsultasConProyeccion(); → método que devuelve el SQL de las consultas con proyección.

Línea 13: `string ObtenerSqlConsultasConFiltroGlobal();` → método que devuelve el SQL de las consultas con filtro global.

Línea 15: `void Agregar(OrdenFabricacion orden);` → método que agrega una orden.

Línea 16: `void Eliminar(OrdenFabricacion orden);` → método que elimina una orden.

Error común: si los métodos devuelven `IQueryable`, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver cadenas con el SQL.

■ Paso 3: Implementar los métodos de análisis de SQL en el repositorio

Modificar `src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs`:

```
public string ObtenerSqlConsultasPendientes()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion);

    return consulta.ToQueryString();
}

public string ObtenerSqlConsultasConInclude()
{
    var consulta = _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Where(o => o.Cliente == "Constructora del Norte");

    return consulta.ToQueryString();
}

public string ObtenerSqlConsultasConProyeccion()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new { o.NumeroOrden, o.Cliente });

    return consulta.ToQueryString();
}

public string ObtenerSqlConsultasConFiltroGlobal()
{
    var consulta = _context.OrdenesFabricacion
        .Include(o => o.Planchas);

    return consulta.ToQueryString();
}
```

Línea 1: `public string ObtenerSqlConsultasPendientes()` → declara el método.

Línea 3: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 4: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 5: `.OrderBy(o => o.FechaCreacion);` → ordena por fecha.

Línea 7: `return consulta.ToQueryString();` → devuelve el SQL sin ejecutarlo.

Línea 10: `public string ObtenerSqlConsultasConInclude()` → declara el método.

Línea 12: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 13: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 14: `.Where(o => o.Cliente == "Constructora del Norte");` → filtra por cliente.

Línea 16: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 19: `public string ObtenerSqlConsultasConProyeccion()` → declara el método.

Línea 21: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 22: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 23: `.Select(o => new { o.NumeroOrden, o.Cliente })`; → proyecta a un tipo anónimo.

Línea 25: `return consulta.ToQueryString()`; → devuelve el SQL.

Línea 28: `public string ObtenerSqlConsultasConFiltroGlobal()` → declara el método.

Línea 30: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 31: `.Include(o => o.Planchas)`; → incluye la colección de planchas.

Línea 33: `return consulta.ToQueryString()`; → devuelve el SQL.

Error común: si se llama a `ToList` antes de `ToQueryString`, la consulta se ejecuta y `ToQueryString` no funciona. Se debe llamar a `ToQueryString` directamente sobre el `IQueryable`.

| Paso 4: Crear el caso de uso de análisis de SQL

Crear el archivo `src/AceriaData.Application/UseCases/AnalisisSqlUseCase.cs`:

```
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class AnalisisSqlUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public AnalisisSqlUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== ANÁLISIS DEL SQL GENERADO ===");

        DemostrarSqlConsultasPendientes();
        DemostrarSqlConsultasConInclude();
        DemostrarSqlConsultasConProyeccion();
        DemostrarSqlConsultasConFiltroGlobal();
    }

    private void DemostrarSqlConsultasPendientes()
    {
        Console.WriteLine("\n--- SQL de consultas pendientes ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasPendientes();
        Console.WriteLine(sql);
    }

    private void DemostrarSqlConsultasConInclude()
    {
        Console.WriteLine("\n--- SQL de consultas con Include ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasConInclude();
        Console.WriteLine(sql);
    }

    private void DemostrarSqlConsultasConProyeccion()
    {
        Console.WriteLine("\n--- SQL de consultas con proyección ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasConProyeccion();
        Console.WriteLine(sql);
    }

    private void DemostrarSqlConsultasConFiltroGlobal()
    {
        Console.WriteLine("\n--- SQL de consultas con filtro global de Soft Delete ---");
        var sql = _unidad.Ordenes.ObtenerSqlConsultasConFiltroGlobal();
    }
}
```

```

        Console.WriteLine(sql);
    }
}

```

Línea 1: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 3: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 5: public class AnalisisSqlUseCase → declara el caso de uso.

Línea 7: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 9: public AnalisisSqlUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 11: _unidad = unidad; → asigna el parámetro al campo.

Línea 14: public void Ejecutar() → declara el método principal.

Línea 16: Console.WriteLine("=== ANÁLISIS DEL SQL GENERADO ==="); → muestra la cabecera.

Línea 18: DemostrarSqlConsultasPendientes(); → llama al método de SQL de consultas pendientes.

Línea 19: DemostrarSqlConsultasConInclude(); → llama al método de SQL con Include.

Línea 20: DemostrarSqlConsultasConProyeccion(); → llama al método de SQL con proyección.

Línea 21: DemostrarSqlConsultasConFiltroGlobal(); → llama al método de SQL con filtro global.

Línea 24: private void DemostrarSqlConsultasPendientes() → declara el método.

Línea 26: Console.WriteLine("\n--- SQL de consultas pendientes ---"); → muestra la cabecera.

Línea 27: var sql = _unidad Ordenes.ObtenerSqlConsultasPendientes(); → llama al método del repositorio.

Línea 28: Console.WriteLine(sql); → imprime el SQL.

Línea 31: private void DemostrarSqlConsultasConInclude() → declara el método.

Línea 33: Console.WriteLine("\n--- SQL de consultas con Include ---"); → muestra la cabecera.

Línea 34: var sql = _unidad Ordenes.ObtenerSqlConsultasConInclude(); → llama al método del repositorio.

Línea 35: Console.WriteLine(sql); → imprime el SQL.

Línea 38: private void DemostrarSqlConsultasConProyeccion() → declara el método.

Línea 40: Console.WriteLine("\n--- SQL de consultas con proyección ---"); → muestra la cabecera.

Línea 41: var sql = _unidad Ordenes.ObtenerSqlConsultasConProyeccion(); → llama al método del repositorio.

Línea 42: Console.WriteLine(sql); → imprime el SQL.

Línea 45: private void DemostrarSqlConsultasConFiltroGlobal() → declara el método.

Línea 47: Console.WriteLine("\n--- SQL de consultas con filtro global de Soft Delete ---"); → muestra la cabecera.

Línea 48: var sql = _unidad Ordenes.ObtenerSqlConsultasConFiltroGlobal(); → llama al método del repositorio.

Línea 49: Console.WriteLine(sql); → imprime el SQL.

Error común: si el método ObtenerSqlConsultasConFiltroGlobal no está implementado, el código no compila. Se debe asegurar que la interfaz y la implementación lo incluyen.

Paso 5: Registrar el caso de uso en el contenedor

Modificar src/AceriaData.Console/Program.cs:

```
services.AddScoped<AnalisisSqlUseCase>();
```

Línea 1: services.AddScoped(); → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<AnalisisSqlUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var useCase = scope.ServiceProvider.GetRequiredService(); → resuelve el caso de uso.

Línea 4: useCase.Ejecutar(); → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción InvalidOperationException.

Paso 7: Insertar datos de prueba con planchas

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes y planchas:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
    context.OrdenesFabricacion.AddRange(orden1, orden2);
    context.SaveChanges();

    var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso = 370.5m, Activa = true };
    var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso = 280.8m, Activa = true };
    var plancha3 = new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso = 125.6m, Activa = true };
    context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3);
    context.SaveChanges();
}
```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: `var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.`

Línea 8: `var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.`

Línea 9: `context.OrdenesFabricacion.AddRange(orden1, orden2); → registra las órdenes.`

Línea 10: `context.SaveChanges(); → inserta las órdenes.`

Línea 12: `var plancha1 = new PlanchaAcero { ... }; → crea la primera plancha.`

Línea 13: `var plancha2 = new PlanchaAcero { ... }; → crea la segunda plancha.`

Línea 14: `var plancha3 = new PlanchaAcero { ... }; → crea la tercera plancha.`

Línea 15: `context.PlanchasAcero.AddRange(plancha1, plancha2, plancha3); → registra las planchas.`

Línea 16: `context.SaveChanges(); → inserta las planchas.`

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con el SQL generado por cada consulta. Se observa el SQL de las consultas pendientes, con Include, con proyección y con filtro global.

Paso 9: Analizar la salida

La salida del programa muestra el SQL generado por cada consulta:

```
=== ANÁLISIS DEL SQL GENERADO ===
```

```
--- SQL de consultas pendientes ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].
[Observaciones], [o].[IsDeleted], [o].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[FechaCreacion]
```

```
--- SQL de consultas con Include ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].
[Observaciones], [o].[IsDeleted], [o].[DeletedAt], [p].[Id], [p].[OrdenId], [p].[Espesor], [p].[Ancho],
[p].[Largo], [p].[Peso], [p].[Activa], [p].[IsDeleted], [p].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[Cliente] = N'Constructora del Norte' AND [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted]
= CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

```
--- SQL de consultas con proyección ---
```

```
SELECT [o].[NumeroOrden], [o].[Cliente]
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[Estado] = N'Pendiente' AND [o].[IsDeleted] = CAST(0 AS bit)
```

```
--- SQL de consultas con filtro global de Soft Delete ---
```

```
SELECT [o].[Id], [o].[NumeroOrden], [o].[Cliente], [o].[FechaCreacion], [o].[Estado], [o].
[Observaciones], [o].[IsDeleted], [o].[DeletedAt], [p].[Id], [p].[OrdenId], [p].[Espesor], [p].[Ancho],
[p].[Largo], [p].[Peso], [p].[Activa], [p].[IsDeleted], [p].[DeletedAt]
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]
```

La primera sección muestra el SQL de las consultas pendientes. La segunda sección muestra el SQL de las consultas con Include. La tercera sección muestra el SQL de las consultas con proyección. La cuarta sección muestra el SQL de las consultas con filtro global.

Observaciones: el SQL de la consulta con proyección solo incluye las columnas proyectadas. El SQL de la consulta con Include incluye todas las columnas de ambas tablas. El filtro global de Soft Delete se aplica automáticamente en todas las consultas.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerSqlConsultasConProyeccion para llamar a ToList antes de ToQueryString:

```
public string ObtenerSqlConsultasConProyeccion()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new { o.NumeroOrden, o.Cliente })
        .ToList();

    return "La consulta ya se ha ejecutado, no se puede obtener el SQL.";
}
```

Resultado esperado: el método devuelve un mensaje indicando que la consulta ya se ha ejecutado. ToQueryString no funciona sobre una lista materializada.

Solución: llamar a ToQueryString directamente sobre el IQueryable, sin materializar.

```
public string ObtenerSqlConsultasConProyeccion()
{
    var consulta = _context.OrdenesFabricacion
        .Where(o => o.Estado == "Pendiente")
        .Select(o => new { o.NumeroOrden, o.Cliente });

    return consulta.ToQueryString();
}
```

Resultado esperado con la solución: el método devuelve el SQL sin ejecutar la consulta.

Errores comunes del ejercicio

Error	Causa	Solución
ToQueryString no funciona	Se llamó a ToList antes	Llamar a ToQueryString sobre el IQueryable
SQL no incluye el filtro	El filtro se aplica en memoria	Aplicar el filtro antes de materializar
SQL no incluye el JOIN	Se olvidó el Include	Añadir el Include
Logging no muestra SQL	Categoría incorrecta o nivel insuficiente	Ajustar categorías y nivel
Datos sensibles expuestos	EnableSensitiveDataLogging habilitado en producción	Deshabilitar en producción
Errores detallados en producción	EnableDetailedErrors habilitado en producción	Deshabilitar en producción

Reto resuelto: Analizar el SQL de una consulta con múltiples Include

Reto: Crear un método en el repositorio que devuelva el SQL de una consulta con Include de planchas, Include de detalle y ThenInclude de aleaciones. Analizar el SQL generado y comprobar si se produce un producto cartesiano.

Solución paso a paso:**| Paso 1: Añadir el método a la interfaz IOrdenRepositorio:**

```
string ObtenerSqlConsultasConMultiplesInclude();
```

Línea 1: string ObtenerSqlConsultasConMultiplesInclude(); → declara el método.

| Paso 2: Implementar el método en OrdenRepositorio:

```
public string ObtenerSqlConsultasConMultiplesInclude()
{
    var consulta = _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .Where(o => o.Estado == "Pendiente");

    return consulta.ToQueryString();
}
```

Línea 1: public string ObtenerSqlConsultasConMultiplesInclude() → declara el método.

Línea 3: var consulta = _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 5: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 6: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 7: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 8: .Where(o => o.Estado == "Pendiente"); → filtra por estado.

Línea 10: return consulta.ToQueryString(); → devuelve el SQL.

| Paso 3: Añadir la demostración en el caso de uso:

```
private void DemostrarSqlConsultasConMultiplesInclude()
{
    Console.WriteLine("\n--- SQL de consultas con múltiples Include ---");
    var sql = _unidad.Ordenes.ObtenerSqlConsultasConMultiplesInclude();
    Console.WriteLine(sql);
}
```

Línea 1: private void DemostrarSqlConsultasConMultiplesInclude() → declara el método.

Línea 3: Console.WriteLine("\n--- SQL de consultas con múltiples Include ---"); → muestra la cabecera.

Línea 4: var sql = _unidad.Ordenes.ObtenerSqlConsultasConMultiplesInclude(); → llama al método del repositorio.

Línea 5: Console.WriteLine(sql); → imprime el SQL.

| Paso 4: Llamar al método desde Ejecutar:

```
DemostrarSqlConsultasConMultiplesInclude();
```

| Paso 5: Ejecutar dotnet run y analizar el SQL.

Resultado esperado: el SQL generado incluye dos LEFT JOIN, uno con la tabla de planchas y otro con la tabla de detalles, más un tercer LEFT JOIN con la tabla de entidades intermedias y un cuarto LEFT JOIN con la tabla de aleaciones. Como se incluyen dos colecciones (planchas y entidades intermedias), EF Core genera un producto cartesiano: cada plancha se combina con cada entidad intermedia. Esto provoca un número elevado de filas. La solución es usar AsSplitQuery, que se estudiará en el punto 4.8.

| Analogía final

El análisis del SQL generado en una acería es como revisar el plano de una orden de búsqueda antes de enviarla al archivo central. En lugar de enviar la orden y ver qué devuelve el archivo, el jefe de planta revisa la orden para asegurarse de que las condiciones son correctas y de que el archivo va a buscar exactamente lo que se necesita. ToQueryString es como ver la orden de búsqueda sin enviarla. El logging es como el registro de todas las búsquedas que se han enviado al archivo, con sus condiciones y sus resultados. Las categorías de logging son como los distintos libros de registro: uno para las búsquedas, otro para las traducciones, otro para las escrituras. Los niveles de logging son como el nivel de detalle del registro: en producción solo se registran los errores, en desarrollo se registra todo. EnableSensitiveDataLogging es como incluir los nombres de los clientes en el registro: útil en desarrollo, pero peligroso en producción. Así funciona el análisis del SQL en EF Core: se inspecciona la consulta antes de ejecutarla, se registra lo que se ejecuta y se detectan problemas antes de que afecten al rendimiento.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de análisis de SQL a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio.

Creado el caso de uso AnalisisSqlUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes y planchas.

Ejecutado las demostraciones de SQL de consultas pendientes, con Include, con proyección y con filtro global.

Analizado la salida y comprobado que el filtro global se aplica automáticamente.

Diagnosticado el error de llamar a ToList antes de ToQueryString.

Creado el método ObtenerSqlConsultasConMultiplesInclude y analizado el producto cartesiano.
